Files in C

Need for files

So far the operations using the C program are done on a prompt/terminal which is not stored anywhere.

The output is deleted when the program is closed.

But in the software industry, most programs are written to store the information fetched from the program.

Advantages of using files

- **Reusability:** The data stored in the file can be accessed, updated, and deleted anywhere and anytime providing high reusability.
- **Portability:** Without losing any data, files can be transferred to another in the computer system. The risk of flawed coding is minimized with this feature.
- **Efficient:** A large amount of input may be required for some programs. File handling allows you to easily access a part of a file using few instructions which saves a lot of time and reduces the chance of errors.
- **Storage Capacity:** Files allow you to store a large amount of data without having to worry about storing everything simultaneously in a program.

Types of files

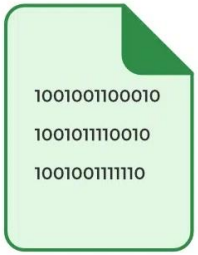
Types of Files in C

A file can be classified into two types based on the way the file stores the data.

- **Text Files**
- **Binary Files**



file.txt



file.bin

1. Text Files

A text file contains data in the **form of ASCII characters** and is generally used to store a stream of characters.

Each line in a text file ends with a new line character (‘\n’).

It can be read or written by any text editor.

They are generally stored with **.txt** file extension.

Text files can also be used to store the source code.

2. Binary Files

A binary file contains data in **binary form (i.e. 0’s and 1’s)** instead of ASCII characters.

They contain data that is stored in a similar manner to how it is stored in the main memory.

The binary files can be created only from within a program and their contents can only be read by a program.

More secure as they are not easily readable.

They are generally stored with **.bin** file extension.

File Operations

C File Operations

- C file operations refer to the different possible operations that we can perform on a file in C such as:
- Creating a new file – **fopen() with attributes as “a” or “a+” or “w” or “w+”**
- Opening an existing file – **fopen()**
- Reading from file – **fscanf() or fgets()**
- Writing to a file – **fprintf() or fputs()**
- Moving to a specific location in a file – **fseek(), rewind()**
- Closing a file – **fclose()**

File Pointer

File Pointer in C

- A file pointer is a reference to a particular position in the opened file. It is used in file handling to perform all file operations such as read, write, close, etc. We use the **FILE** macro to declare the file pointer variable. The FILE macro is defined inside **<stdio.h>** header file.
- **Syntax of File Pointer**
- **FILE*** *pointer_name*;
  File Pointer is used in almost all the file operations in C.

Opening a file

Open a File in C

- For opening a file in C, the fopen() function is used with the filename or file path along with the required access modes.

Syntax of fopen()

```
FILE *fptr;
fptr = fopen(filename, mode);
```

Parameters

- *file_name:* name of the file when present in the same directory as the source file. Otherwise, full path. The name of the actual file you want to open (or create), like filename.txt
- *mode:* Specifies for what operation the file is being opened. (read, write or append):

  w - Writes to a file
  a - Appends new data to a file
  r - Reads from a file

Return Value

- If the file is opened successfully, returns a file pointer to it.
- If the file is not opened, then returns NULL.

File Opening Modes

Mode	Description
r	Opens an existing text file for reading purposes.
w	Opens a text file for writing. If it does not exist, then a new file is created. Here your program will start writing content from the beginning of the file.
a	Opens a text file for writing in appending mode. If it does not exist, then a new file is created. Here your program will start appending content in the existing file content.
r+	Opens a text file for both reading and writing.
	Opens a text file for both reading and writing. It first truncates the file to zero length if it exists, otherwise creates a file if it does not exist.
a+	Opens a text file for both reading and writing. It creates the file if it does not exist. The reading will start from the beginning but writing can only be appended.

Create a File

To create a file, you can use the **w** mode inside the fopen() function.

- The w mode is used to write to a file.
- **However**, if the file does not exist, it will create one for you:

Example

```
#include <stdio.h>
#include <stdlib.h>
void main()
{
   FILE *fptr;

   // Create a file
   fptr = fopen("filename.txt", "w");

   // Close the file
   fclose(fptr);
}
```

If you want to create the file in a specific folder, just provide an absolute path (remember to use double backslashes to create a single backslash (\), like
fptr = fopen("C:\\directoryname\\filename.txt", "w");

Closing the file (**fclose()**)
It is considered as good practice, because it makes sure that:
Changes are saved properly
Other programs can use the file (if you want)
Clean up unnecessary memory space

Note: The file is created in the same directory as your other C files, if nothing else is specified.

Write To a File

Use the **w** mode to write something to the file we created.

- The w mode means that the file is opened for **writing**. To insert content to it, you can use the **fprintf()** function and add the pointer variable (fptr in our example) and some text:

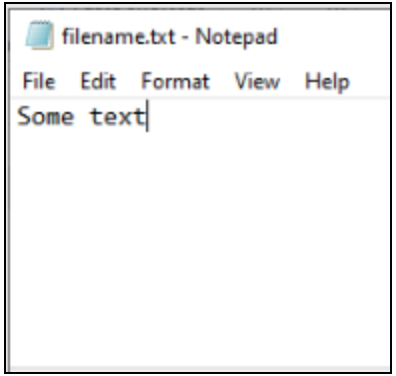
Example

```
#include <stdio.h>
#include <stdlib.h>
void main()
{
    FILE *fptr;

    // Open a file in writing mode
    fptr = fopen("filename.txt", "w");

    // Write some text to the file
    fprintf(fptr, "Some text");

    // Close the file
    fclose(fptr);
}
```



- **fputc()** : Writes a single character to a file.
- **fputs()**: Writes a string to a file.
- **fprintf()**: Writes a formatted string (data) to a file.

Note: If you write to a file that already exists, the old content is deleted, and the new content is inserted. This is important to know, as you might accidentally erase existing content.

Try

```
fprintf(fptr, "Hello World!");
```

in the same program and run again. You can see that the file has Hello World" instead of "Some text"

Append To a File

Append Content To a File

If you want to add content to a file without deleting the old content, you can use the a mode.

- The **a** mode appends content at the end of the file:

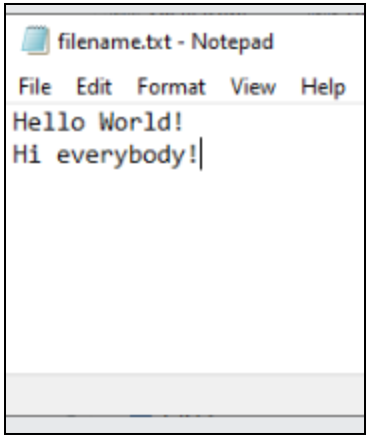
Example:

```
#include <stdio.h>
#include <stdlib.h>
void main()
{
    FILE *fptr;

    // Open a file in append mode
    fptr = fopen("filename.txt", "a");

    // Append some text to the file
    fprintf(fptr, "\nHi everybody!");

    // Close the file
    fclose(fptr);
}
```



Note: Just like with the `w` mode; if the file does not exist, the `a` mode will create a new file with the "appended" content.

Read from a File

Read a File

- To **read** from a file, you can use the r mode:

Example

```
#include <stdio.h>
#include <stdlib.h>
void main()
{
    FILE *fptr;

    // Open a file in read mode
    fptr = fopen("filename.txt", "r");

    // Store the content of the file
    char myString[100];

    // Read the content and store it inside myString
    fgets(myString, 100, fptr);

    // Print the file content
    printf("%s", myString);

    // Close the file
    fclose(fptr);
}
```

- **fgetc()**: Reads a single character from a file.
- **fgets()**: Reads a string from a file.
- **fscanf()**: Reads a formatted string from a file.

fgets (myString, 100, fptr); / takes 3 parameters

1.The first parameter specifies where to store the file content, which will be in the myString array we just created.
2.The second parameter specifies the maximum size of data to read, which should match the size of myString (100).
3.The third parameter requires a file pointer that is used to read the file (fptr in our example).

Note: The fgets function only reads the first line of the file. If you remember, there were two lines of text in filename.txt.

Read from a File

To read every line of the file, you can use a while loop:

Example

```
#include <stdio.h>
#include <stdlib.h>
void main()
{
    FILE *fptr;

    // Open a file in read mode
    fptr = fopen("filename.txt", "r");

    // Store the content of the file
    char myString[100];

    // Read the content and print it
    while(fgets(myString, 100, fptr))
        {
            printf("%s", myString);
        }

    // Close the file
    fclose(fptr);
}
```

Output:

Hello World!
Hi everybody!

If you try to open a file for reading that does not exist, the `fopen()` function will return `NULL`.

we can use an if statement to test for NULL, and print some text instead (when the file does not exist):

Read from a File

If the file exist, read the content and print it.
If the file does not exist, print a message:

Example

```
#include <stdio.h>
#include <stdlib.h>
void main()
{
    FILE *fptr;

    // Open a file in read mode
    fptr = fopen("filename.txt", "r");

    // Store the content of the file
    char myString[100];

    // If the file exist
    if(fptr != NULL)
    {
      // Read the content and print it
      while(fgets(myString, 100, fptr))
     {
        printf("%s", myString);
      }

    // If the file does not exist
    } else {
      printf("Not able to open the file.");
    }

    // Close the file
    fclose(fptr);
}
```

Output:

```
Hello World!
Hi everybody!
```